AI Powered Debt Relief & Financial Recovery Platform Project

Description:

The **AI Powered Debt Relief & Financial Recovery Platform** is an intelligent web application designed to help individuals manage their debts, improve financial planning, and achieve financial stability. The platform allows users to securely register, track their income, expenses, loans, and EMIs, while providing a centralized dashboard to monitor their overall financial status. Its user-friendly interface makes financial management simple, efficient, and accessible.

The platform uses **Artificial Intelligence (AI)** to analyze users' financial data and generate personalized debt repayment strategies, budgeting suggestions, and financial insights. It also provides expense tracking, repayment reminders, and visual reports to help users make informed financial decisions. Built using **React.js, Node.js, Express.js, and MongoDB**, the system ensures secure data management while offering a smart and effective solution for debt relief and long-term financial recovery.

Scenarios

Scenario 1: User Registration
A new user creates an account by entering personal details such as name, email, phone number, and password. After successful registration, the user logs in to access the platform.

Scenario 2: User Login
A registered user logs in securely using their email and password. After authentication, the user is redirected to the dashboard.

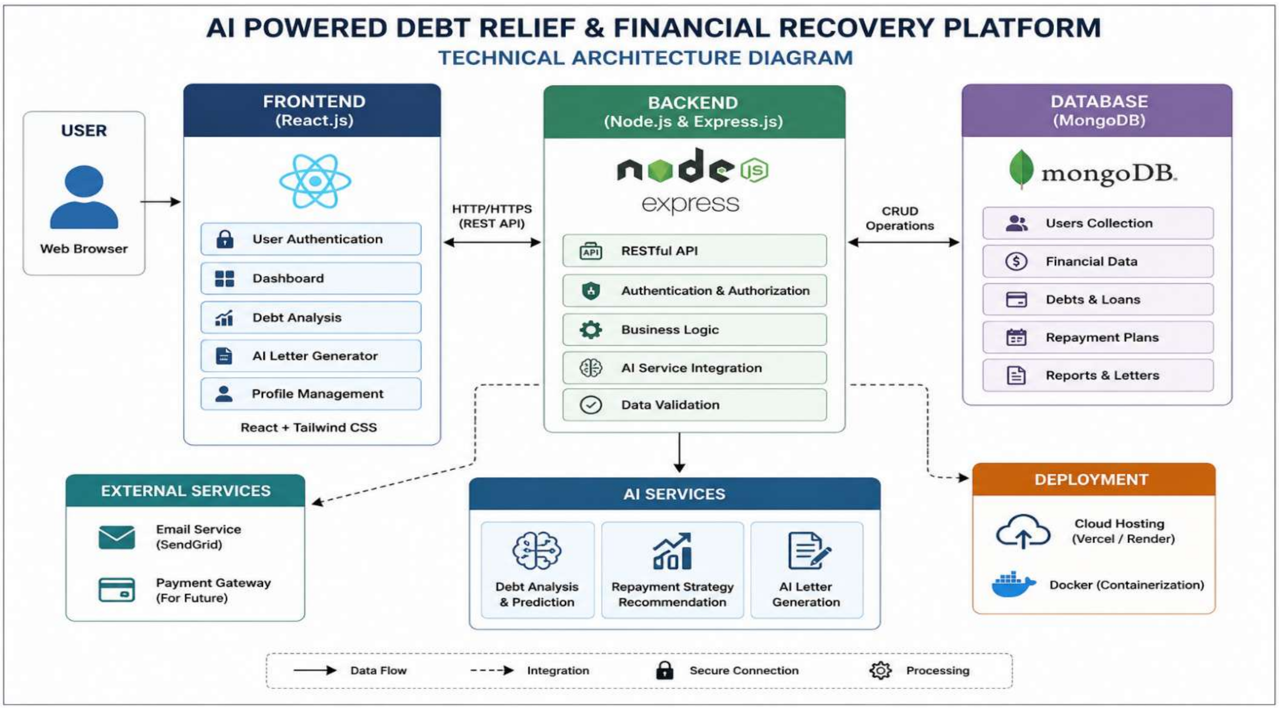
Scenario 3: Debt Analysis
The user enters financial details such as income, monthly expenses, loan amount, interest rate, and EMIs. The AI analyzes this information and provides personalized debt repayment strategies and financial recommendations.

Scenario 4: AI Letter Generation
The user provides the required information, and the platform generates a professional debt relief request letter that can be shared with banks or financial institutions.

Scenario 5: Profile Management
The user can view and update personal information, change account details, and securely manage their profile.

Scenario 6: Dashboard Monitoring
The dashboard displays a summary of the user's financial information, debt status, AI recommendations, and repayment progress, helping users make informed financial decisions.

Technical Architecture:



Description: The **AI Powered Debt Relief & Financial Recovery Platform** utilizes a modular three-tier architecture to provide intelligent debt analysis, financial recovery planning, and AI-assisted debt negotiation support. The frontend is developed using **React.js** with **Tailwind CSS**, offering a responsive and user-friendly interface. The **Node.js** and **Express.js** backend handles authentication, business logic, API routing, and communication with the database. **MongoDB** securely stores user information, debt records, repayment plans, and generated reports. AI services are integrated to analyze financial data, recommend personalized repayment strategies, and generate professional debt negotiation letters. The application is deployed using cloud platforms such as **Vercel** (frontend) and **Render** (backend), ensuring secure and scalable access.

Pre-requisites:

- React.js and Tailwind CSS Fundamentals
- Node.js and Express.js Development
- MongoDB Database Knowledge
- REST API Development
- JavaScript (ES6+) Programming
- AI API Integration (OpenAI/Groq/Gemini or the AI service used)
- Git and GitHub Version Control
- Deployment using Vercel and Render
- VS Code Development Environment

Project Workflow:

Milestone 1: Project Architecture & Environment Setup

- Activity 1.1: Design the overall architecture of the Debt Relief Platform, defining interactions between the frontend, backend, database, and AI services.
- Activity 1.2: Configure the development environment by installing React.js, Node.js, Express.js, MongoDB, and required dependencies.
- Activity 1.3: Set up MongoDB collections for users, debts, repayment plans, and financial reports.
- Activity 1.4: Integrate the selected AI service for debt analysis and financial recommendation generation.

Milestone 2: Backend Development

- Activity 2.1: Develop REST APIs for user authentication, debt management, repayment plans, and financial records.
- Activity 2.2: Implement business logic for debt analysis, repayment calculations, and AI-powered recommendations.
- Activity 2.3: Connect the backend with MongoDB to perform secure CRUD operations.

Milestone 3: AI Integration

- Activity 3.1: Integrate AI services to analyze user financial data and generate personalized debt recovery strategies.
- Activity 3.2: Develop AI-powered debt negotiation letter generation.
- Activity 3.3: Implement AI-based financial insights and repayment suggestions.

Milestone 4: Frontend Development

- Activity 4.1: Design a responsive user interface using React.js and Tailwind CSS.
- Activity 4.2: Develop pages for user registration, login, dashboard, debt management, repayment tracking, AI recommendations, and profile management.
- Activity 4.3: Connect the frontend with backend APIs to display real-time financial data and AI-generated results.

Milestone 5: Testing & Deployment

- Activity 5.1: Perform unit testing, integration testing, and user acceptance testing to ensure application reliability.
- Activity 5.2: Deploy the frontend using Vercel and the backend using Render.
- Activity 5.3: Verify end-to-end functionality, database connectivity, and AI response accuracy before final release.

Milestone 6: Conclusion

The AI Powered Debt Relief & Financial Recovery Platform provides an intelligent and user-friendly solution for individuals facing financial challenges. By integrating artificial intelligence with modern web technologies, the platform helps users analyze their debts, receive personalized repayment strategies, and generate professional debt negotiation letters. This reduces the complexity of financial planning and enables users to make informed decisions for effective debt management.

Milestone 1: Project Architecture and Development Environment Setup

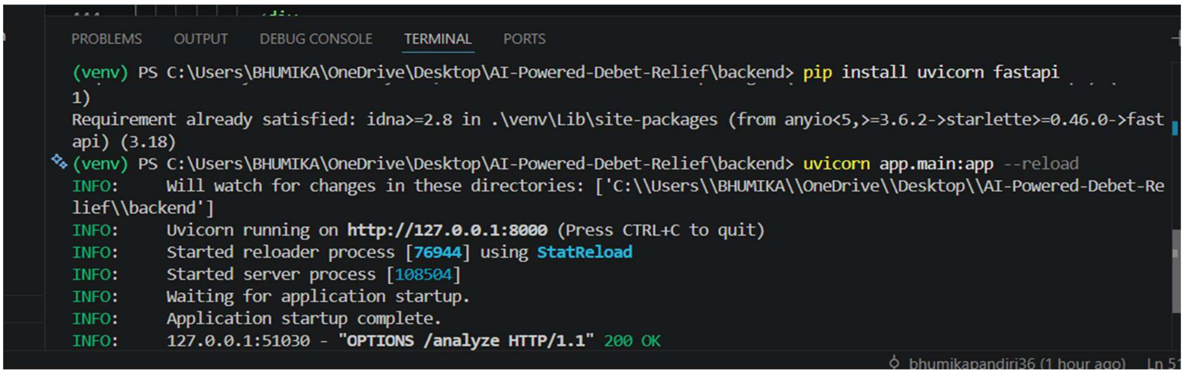
In this milestone, the overall architecture of the AI Powered Debt Relief & Financial Recovery Platform is designed, and the required development environment is configured. The React frontend, FastAPI backend, SQLite database, and Groq AI API are integrated to work together, enabling secure user management, debt analysis, AI-powered financial recommendations, and debt settlement letter generation.

Activity 1.1: Set Up the Development Environment

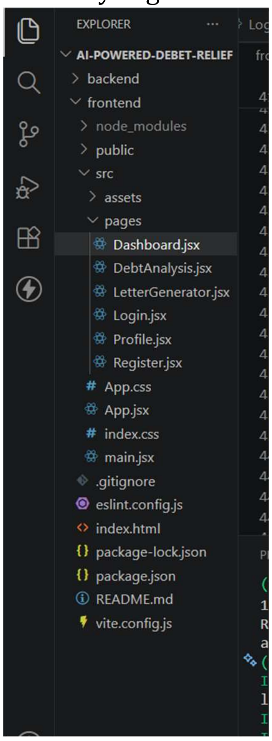
Step 1: Install Python, Node.js, and Visual Studio Code.

Step 2: Create the React frontend and FastAPI backend for the application.

Step 3: Configure the SQLite database and install all required project dependencies, including FastAPI, Uvicorn, Groq SDK, python-dotenv, and frontend packages.
Step 4: Integrate the Groq AI API, configure the environment variables, and verify that the frontend and backend run successfully, ensuring the AI-powered debt analysis and financial recovery features work correctly.

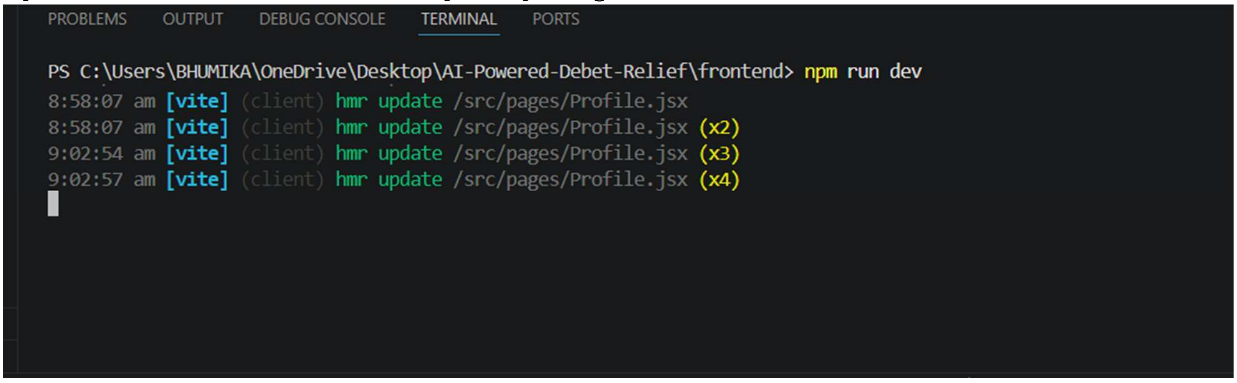


Step 2 – Open the Project in Visual Studio Code
Open Visual Studio Code and load the AI Powered Debt Relief & Financial Recovery Platform project folder. Verify that both the frontend and backend folders are available and the project structure is correctly organized before starting development.



Step 3 – Install Project Dependencies

Open the terminal and install the required packages.



Step 4 – API Key: Enter a Display Name (e.g., Debt Relief AI Platform or Financial Recovery Assistant). Setting an Expiration date is optional. Click Submit to generate the API key. Once the key is created, copy it immediately and store it securely, as it will not be displayed again after you close the dialog. This API key will be used to connect the AI service with the AI Powered Debt Relief & Financial Recovery Platform for features such as AI-based debt analysis, personalized financial recommendations, and smart repayment planning.

Step 5 — Add the Key to Your Project:

Create a **.env** file in the root directory of your project (if it does not already exist) and paste the key as shown below:
GROQ_API_KEY=your_copied_api_key_here

Step 6 — Verify the Key is Loaded:
The app.py file loads this key automatically at startup using python-dotenv:

- from dotenv import load_dotenv
- load_dotenv()
-
- client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

The application uses this API key to securely connect with the Groq API and generate AI-powered debt analysis, financial recovery strategies, budgeting suggestions, and personalized repayment recommendations based on the user's financial information.

Add **.env** to your **.gitignore** file to prevent accidentally pushing the key to a public repository:

.env

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select the best model for the project:

- Understand the Project Requirements: Review the specific needs of the AI Powered Debt Relief & Financial Recovery Platform, focusing on generating personalized debt analysis, budgeting recommendations, EMI repayment strategies, financial recovery plans, and intelligent financial guidance based on user-provided financial information.

- Explore Groq's Model Documentation: Visit the Groq documentation to examine the available LLM models, including their reasoning capabilities, response quality, context windows, processing speed, token limits, and suitability for financial advisory and recommendation tasks.

- Evaluate Model Performance: Compare different models based on the accuracy of financial recommendations, response quality, reasoning ability, inference speed, consistency, and their capability to provide structured and easy-to-understand debt recovery suggestions.

- Select the Optimal Model: Choose llama-3.3-70b-versatile for its excellent balance of reasoning capability, high-quality responses, fast generation speed, and ability to provide detailed financial guidance. Configure the model with a temperature of 0.7 and max_tokens of 2048 to generate informative, personalized, and reliable financial recommendations.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components such as the React frontend, Flask backend, SQLite database, Groq AI API, and the communication flow between users, backend services, and AI modules.
- **Detail Frontend Functionality:** Outline how users interact with the application by creating an account, logging in, entering financial details such as income, expenses, loans, EMI information, and financial goals, then requesting AI-powered debt analysis and personalized recovery recommendations through an interactive dashboard.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming API requests, validates and sanitizes user inputs, stores financial information securely in the SQLite database, performs required financial calculations, constructs AI prompts, communicates with the Groq API, and sends structured responses back to the frontend.
- **Describe AI Integration Points:** Define how the backend creates structured prompts using the user's financial profile, debt information, repayment history, monthly

income, expenses, and financial goals, sends the prompt to the Groq API, processes the AI-generated recommendations, and returns personalized debt analysis, budgeting strategies, repayment plans, and financial recovery suggestions to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure **Python 3.10+** is installed on your system along with **pip** for installing and managing project dependencies.
- **Create a Virtual Environment:** Set up a virtual environment using **venv** to isolate project dependencies.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debet-Relief> C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debt-Relief> python -m venv venv
>>
>> C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debt-Relief> venv\Scripts\activate
>>
>> (venv) C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debt-Relief> pip install -r requirements.txt
```

Install FastAPI: Use pip to install **FastAPI**, which is used to develop the backend REST APIs for handling user authentication, loan management, debt analysis, and AI requests.

```
(venv) C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debt-Relief> pip install fastapi
```

Install Uvicorn: Install **Uvicorn** as the ASGI server to run the FastAPI application.

```
(venv) C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debt-Relief> pip install uvicorn
```

Install Google Generative AI SDK: Install the **Google Generative AI** library to integrate the Gemini AI model for debt analysis, financial recovery recommendations, and settlement letter generation.

```
(venv) C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debt-Relief> pip install google-generativeai
```

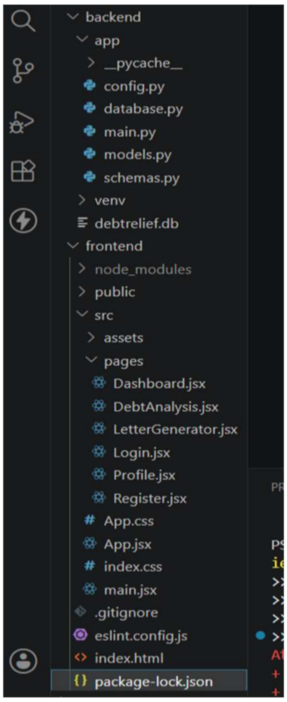
Install python-dotenv: Install **python-dotenv** to securely load environment variables such as the Gemini API key.

```
(venv) C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debt-Relief> pip install python-dotenv
```

Configure the API Key: Create a **.env** file in the project root directory and add your Gemini API key.

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Set Up Application Structure: Create the project directory structure including the **frontend** (React application), **backend** (FastAPI application), **SQLite database**, configuration files, **requirements.txt**, **package.json**, **README.md**, and other supporting files required for implementing the AI Powered Debt Relief & Financial Recovery Platform.



Milestone 2: Core Functionalities Development

In this milestone, the core functionalities of the AI Powered Debt Relief & Financial Recovery Platform are developed to provide users with secure account management, AI-powered debt analysis, personalized financial recovery recommendations, debt settlement letter generation, and profile management. These features enable users to manage their financial information effectively and receive intelligent guidance for improving their financial health.

Activity 2.1: Develop Core Platform Features

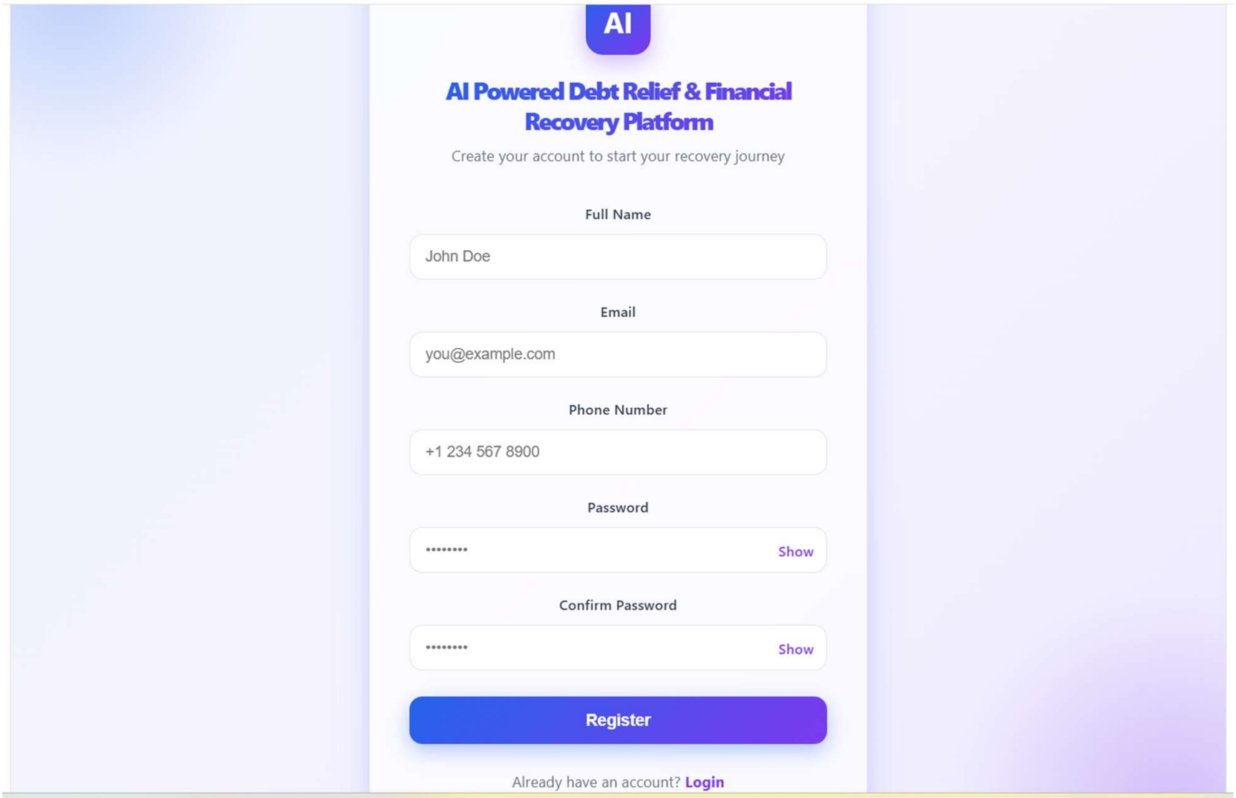
Implement the core functionalities of the AI Powered Debt Relief & Financial Recovery Platform to assist users in managing their debts and planning financial recovery.

- **User Authentication**: Develop secure user registration and login functionality to protect user accounts and ensure authorized access to the platform.

- **AI-Powered Debt Analysis**: Enable users to enter financial details such as income, expenses, loans, and EMI information. The system analyzes the data using the Gemini AI model and provides personalized debt analysis, repayment strategies, and budgeting recommendations.

- **AI Debt Settlement Letter Generator:** Allow users to automatically generate professional debt negotiation and settlement letters based on their financial information and debt details using AI.

- **Profile Management:** Provide users with the ability to view, edit, and update their personal information, financial profile, and account details securely.



- # Milestone 3: Backend Application Development
- Milestone 3 focuses on developing the core backend logic of the AI Powered Debt Relief & Financial Recovery Platform. In this phase, FastAPI REST APIs are created to handle user authentication, debt analysis, AI-powered financial recommendations, debt settlement letter generation, and profile management. The backend processes user requests, communicates with the SQLite database, integrates with the Gemini AI API, and returns appropriate responses to the frontend.

- **Activity 3.1: Develop the Main Backend Logic**
- Define the Core API Routes
- Create the backend routes to perform the following operations:
- User Registration – Register new users securely.
- User Login – Authenticate users and provide secure access.
- Debt Analysis – Accept financial details and generate AI-powered debt analysis and repayment recommendations.
- AI Letter Generator – Generate professional debt settlement and negotiation letters using Gemini AI.
- Profile Management – Retrieve and update user profile information.
- Process User Requests
- Receive financial information submitted from the React frontend.
- Validate user inputs before processing.
- Store and retrieve user information from the SQLite database.
- Generate AI-powered debt analysis and settlement letters using the Gemini API.
- Return structured JSON responses to the frontend.

```python
from sqlalchemy import Column, Integer, String
from .database import Base


class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    full_name = Column(String, nullable=False)
    email = Column(String, unique=True, index=True)
    phone = Column(String)
    monthly_income = Column(Integer)
    monthly_expenses = Column(Integer)
    total_debt = Column(Integer)
```

```python
@app.get("/")
def home():
    return {
        "message": "Welcome to AI Powered Debt Relief & Financial Recovery Platform"
    }


@app.post("/register")
def register(user: UserCreate, db: Session = Depends(get_db)):
    new_user = User(
        full_name=user.full_name,
        email=user.email,
        phone=user.phone,
        monthly_income=user.monthly_income,
        monthly_expenses=user.monthly_expenses,
        total_debt=user.total_debt
    )

    db.add(new_user)
    db.commit()
    db.refresh(new_user)
```

```python
from pydantic import BaseModel

class UserCreate(BaseModel):
    full_name: str
    email: str
    phone: str
    monthly_income: int
    monthly_expenses: int
    total_debt: int


class DebtAnalysisRequest(BaseModel):
    monthly_income: int
    monthly_expenses: int
    total_debt: int
```

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///./debtrelief.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debet-Relief> cd backend
PS C:\Users\BHUMIKA\OneDrive\Desktop\AI-Powered-Debet-Relief\backend> python -m uvicorn app.main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\BHUMIKA\\OneDrive\\Desktop\\AI-Powered-Debet-Re
lief\\backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [53952] using StatReload
INFO:     Started server process [50976]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Set Up Route Handlers for Each Feature

- Implement FastAPI route handlers for user registration, login, debt analysis, AI letter generation, and profile management.
- Validate all incoming requests using Pydantic models before processing.
- Connect backend APIs with the SQLite database to perform CRUD operations.
- Send financial information to the Gemini AI model and return personalized recommendations and generated settlement letters.
- Return appropriate HTTP status codes and JSON responses for successful requests and error handling.

- **Define AI Prompt Templates:**
- **DEBT_ANALYSIS_PROMPT**: Defines the instructions for generating personalized debt analysis and financial recovery recommendations based on the user's financial information.

```python
@app.get("/")
def home():
    return {
        "message": "Welcome to AI Powered Debt Relief & Financial Recovery Platform"
    }


@app.post("/register")
def register(user: UserCreate, db: Session = Depends(get_db)):
    new_user = User(
        full_name=user.full_name,
        email=user.email,
        phone=user.phone,
        monthly_income=user.monthly_income,
        monthly_expenses=user.monthly_expenses,
        total_debt=user.total_debt
    )

    db.add(new_user)
    db.commit()
    db.refresh(new_user)
```

- **LETTER_GENERATION_PROMPT**: Defines the instructions for generating professional debt settlement and negotiation letters using the user's financial information.

```javascript
export default function LetterGenerator() {
  const handleGenerateLetter = () => {
    const letter = `Date: ${today}

I can be reached at ${email || "[Email Address]"} for any further documentation or discussion regarding t

Sincerely,
${fullName || "[Full Name]"}`;

    setGeneratedLetter(letter);
  };

  const handleCopyLetter = () => {
    if (!generatedLetter) return;
    navigator.clipboard.writeText(generatedLetter).then(() => {
      setCopySuccess(true);
      setTimeout(() => setCopySuccess(false), 2000);
    });
  };

  const handleDownloadLetter = () => {
    if (!generatedLetter) return;
    const blob = new Blob([generatedLetter], { type: "text/plain" });
```

- **build_prompt():** Creates a structured AI prompt by combining the user's financial information with predefined instructions before sending it to the Gemini AI model.

```
export default function DebtAnalysis() {

  const handleAnalyzeDebt = async () => {
    setError("");
    try {
      setLoading(true);
      const response = await fetch("http://127.0.0.1:8000/analyze", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          full_name: fullName,
          email: email,
          phone: phone,
          monthly_income: monthlyIncome,
          monthly_expenses: monthlyExpenses,
          total_debt: totalDebt,
        }),
      });
      const data = await response.json();
      setResult(data);
    } catch (err) {
      console.error(err);
      setError("Unable to analyze debt. Please try again.");
    } finally {
```

- **Integrate the Debt Analysis and Letter Generation Logic in Each Function**
- **Implement analyze_debt():** Process the user's financial information, including monthly income, monthly expenses, and total debt. Calculate the disposable income and debt ratio, determine the financial risk level, and generate personalized debt management recommendations before returning the results to the frontend.
- **Implement handleGenerateLetter():** Generate a professional debt negotiation and settlement letter by combining the user's financial details, including full name, bank name, loan type, outstanding amount, hardship reason, and requested relief, into a structured letter template. The generated letter is displayed to the user and can be copied or downloaded for future use.
- **Validate User Input:** Validate the user's financial information, including monthly income, monthly expenses, total debt, and personal details, before processing the request. This ensures that all required fields are provided and the calculations are performed accurately.
- **Home Route (/):** Displays a welcome message confirming that the AI Powered Debt Relief & Financial Recovery Platform backend is running successfully.
- **Register Route (/register):** Accepts user registration details, stores them in the SQLite database, and returns a success message with the generated user ID.
- **Users Route (/users):** Retrieves and returns the list of registered users from the database.
- **Debt Analysis Route (/analyze):** Accepts the user's financial information, calculates the disposable income and debt ratio, determines the financial risk level, and returns personalized financial advice as a JSON response.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a modern, responsive, and user-friendly interface for the AI Powered Debt Relief & Financial Recovery Platform. The frontend is built using React.js and provides interactive pages for user registration, debt analysis, AI-based debt negotiation letter generation, dashboard navigation, and profile management.

Activity 4.1: Designing and Developing the User Interface

Develop the React-Based User Interface

- Create separate React components and pages for Login, Registration, Dashboard, Debt Analysis, Letter Generator, and Profile to provide a modular and maintainable application structure.
- Design the interface using modern UI principles with responsive layouts, gradient backgrounds, glassmorphism-inspired cards, and intuitive navigation to improve user experience.
- Use reusable React components, icons, and consistent typography to maintain a professional and visually appealing interface across all pages.

Design a Responsive Layout

- Implement responsive layouts using CSS Flexbox and CSS Grid to ensure the application adapts smoothly across desktop, tablet, and mobile devices.
- Apply consistent spacing, color themes, hover effects, and responsive form controls to enhance usability.
- Use media queries to optimize the layout for different screen sizes and improve accessibility.

Create Separate UI Components for Each Feature

Registration Form: Collects user details including full name, email, phone number, monthly income, monthly expenses, and total debt.

Debt Analysis Page: Accepts the user's financial information and displays the calculated debt ratio, disposable income, financial risk level, and personalized financial advice.

Debt Negotiation Letter Generator: Allows users to enter their financial details and automatically generates a professional debt negotiation letter, with options to copy or download the generated letter.

Dashboard: Provides quick navigation to all major features of the platform.

- Profile Page: Displays and manages the user's personal and financial information.

Activity 4.2: Creating Dynamic Functionality with React

Handle User Interactions

- Use React state management (useState) to capture user inputs, validate form data, and update the interface dynamically based on user interactions.
- Handle button click events to perform debt analysis and generate professional debt negotiation letters without reloading the page.

Render Results Dynamically

- Display the calculated debt analysis results, including debt ratio, disposable income, financial risk level, and financial recommendations immediately after processing the user's information.
- Dynamically generate and display the debt negotiation letter using the user-provided financial information.
- Update the interface instantly whenever the user modifies the input values.

Manage Application State

- Maintain application state using React Hooks to manage user inputs, generated debt analysis results, generated letters, and UI feedback.
- Provide visual feedback during user interactions and ensure the interface remains responsive and consistent throughout the application.

Milestone 5: Local Execution and Testing

In Milestone 5, the focus is on running the **AI Powered Debt Relief & Financial Recovery Platform** in the local development environment and verifying that all modules function correctly. The application was tested using the React frontend and FastAPI backend to ensure proper communication, data processing, and user interaction. Since public deployment was not required for this project, the application was executed and validated locally.

Activity 5.1: Preparing the Application for Local Execution

Set Up the Backend Environment

- Open a new terminal in Visual Studio Code.
- Navigate to the backend folder.
- Create and activate a Python virtual environment.

```
cd backend
python -m venv venv
```

Activate the virtual environment.

```
venv\Scripts\activate
```

Install the required dependencies.

```
pip install -r requirements.txt
```

Configure the Application

- Create a **.env** file if required.
- Add the necessary application configuration such as the database path or API key (if applicable).
- Ensure the SQLite database and backend configuration files are correctly initialized before running the application.

Activity 5.2: Local Testing and Verification

Start the FastAPI Backend

Run the backend server using Uvicorn.

python -m uvicorn app.main:app –reload

The backend runs at:

http://127.0.0.1:8000

Start the React Frontend

Open another terminal.

cd frontend

npm install

npm run dev

Verify Application Functionality

Test the following features:

- User Registration
- Dashboard Navigation
- Debt Analysis
- Debt Negotiation Letter Generation
- Profile Management

Verify that:

- User information is stored successfully.
- Debt ratio and disposable income are calculated correctly.
- Personalized financial advice is displayed.
- Debt negotiation letters are generated successfully.
- The frontend communicates correctly with the FastAPI backend.

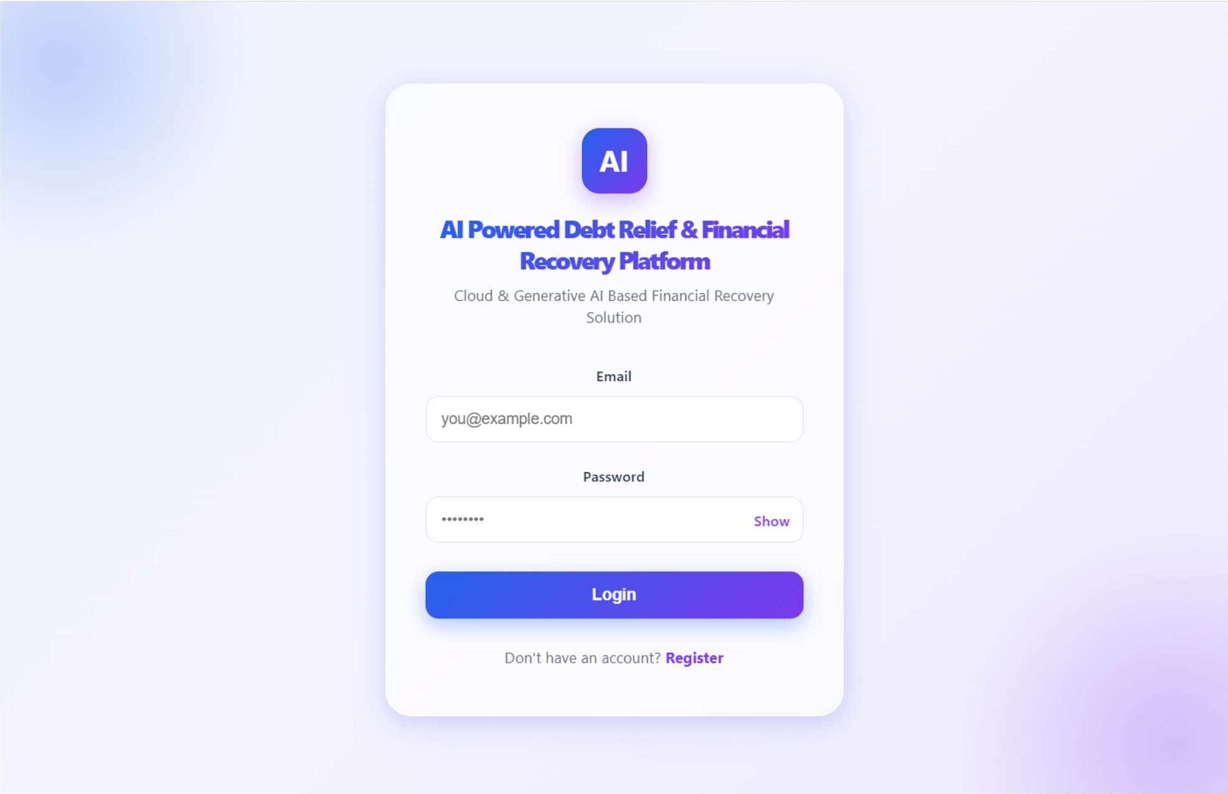
Activity 5.3: Final System Validation

The application was successfully executed and tested in the local development environment. All frontend pages and backend APIs functioned correctly without runtime errors. The system demonstrated successful data processing, financial analysis, and debt negotiation letter generation.

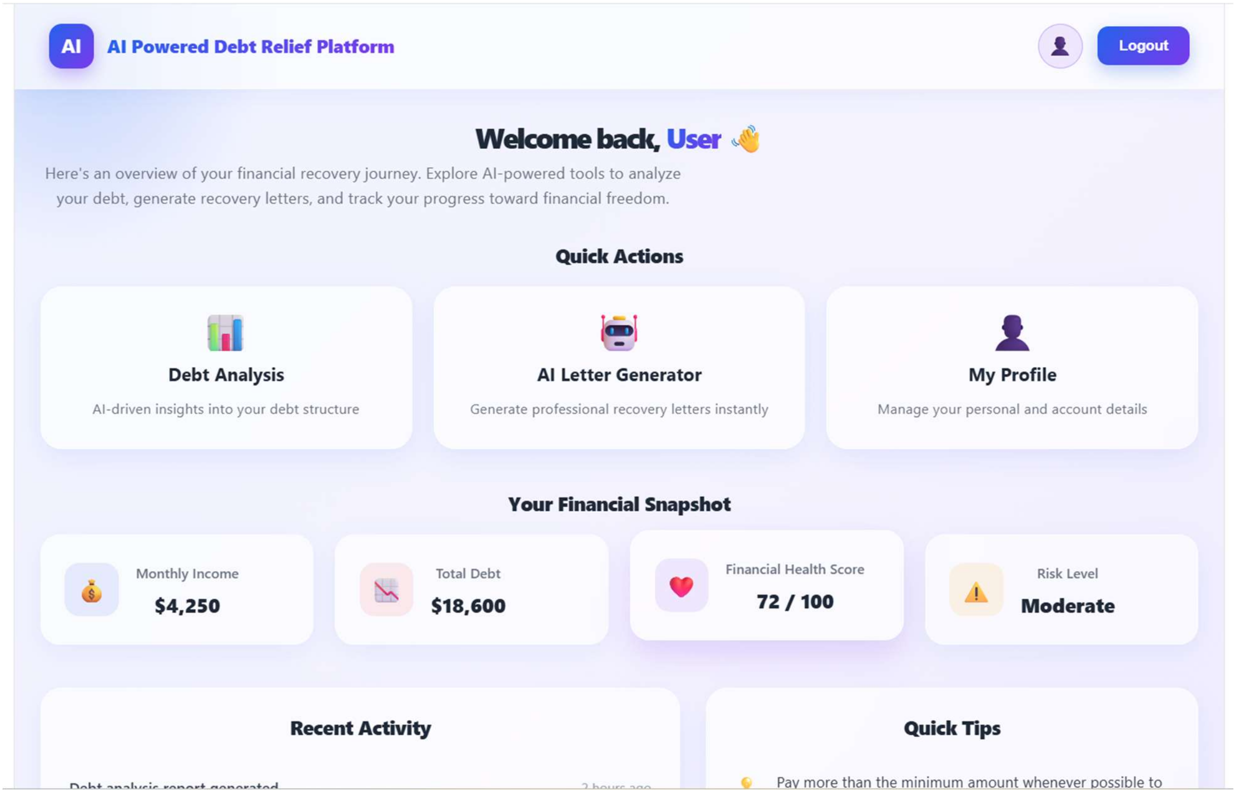
Since public deployment was **not included in the project requirements**, the application was validated entirely within the local development environment.

Exploring the Website's Web Pages:

Home / Generator Page:

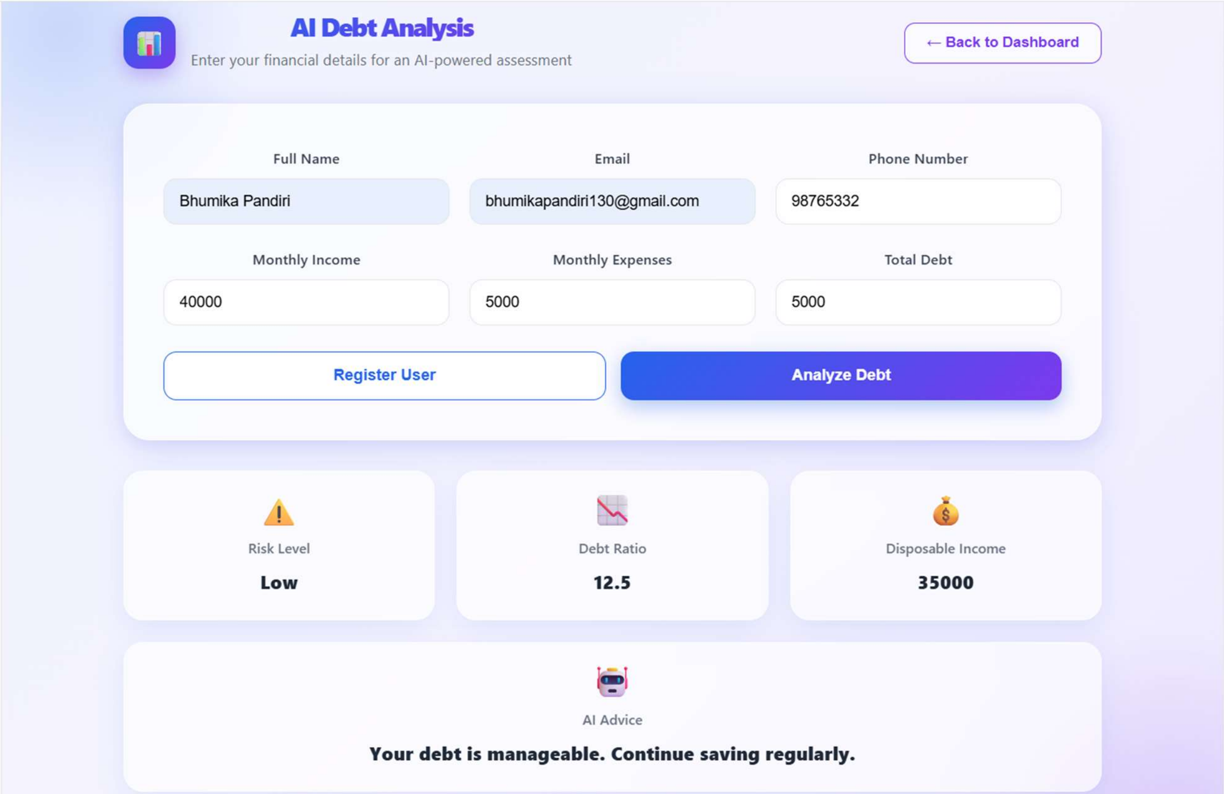


Description:
The Login page serves as the entry point to the **AI Powered Debt Relief & Financial Recovery Platform**. It provides a clean and user-friendly interface where users can securely log in using their email address and password. New users can navigate to the registration page using the **Register** option. The interface is designed with a modern card-based layout, ensuring an intuitive and responsive user experience.



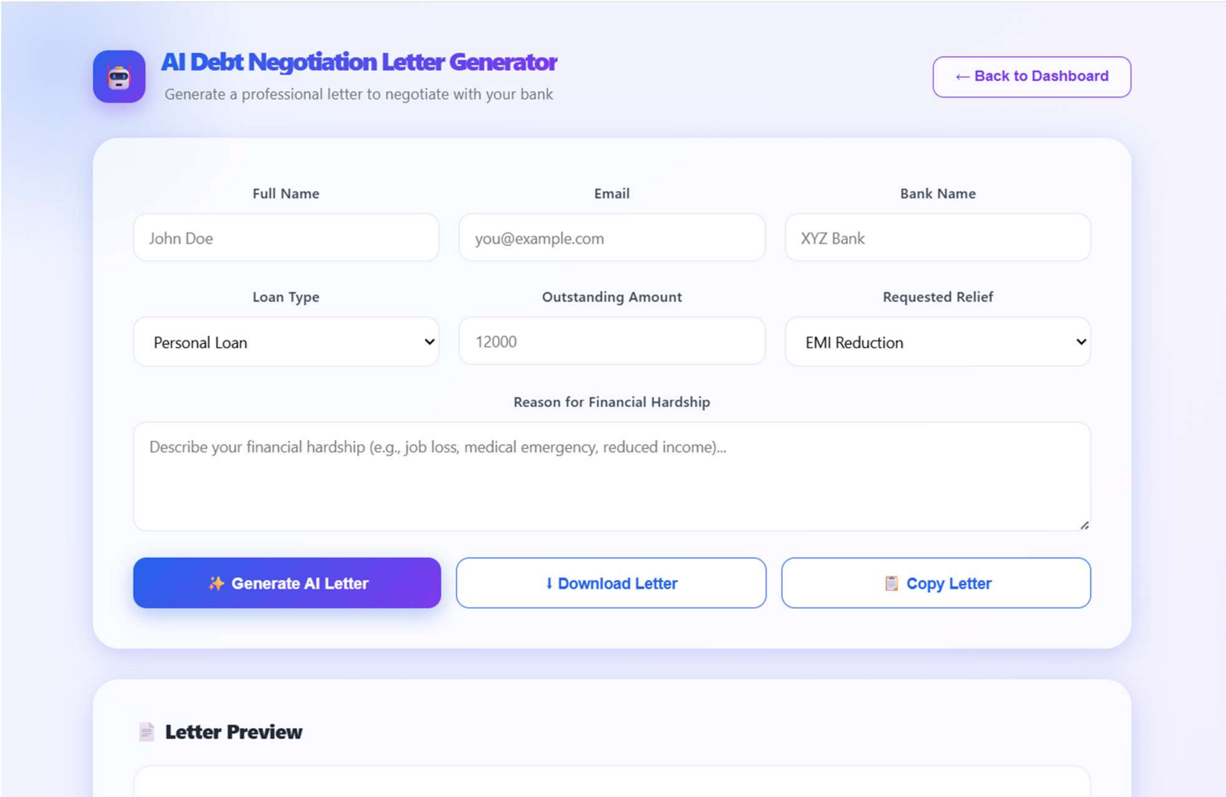
Description:
The Dashboard provides users with a comprehensive overview of their financial recovery journey. It displays quick access cards for Debt Analysis, AI Letter Generator, and My Profile. The dashboard also presents a financial snapshot, including monthly income, total debt, financial health score, and current risk level. Additionally, recent activities and financial tips help users monitor their progress

and make informed financial decisions.



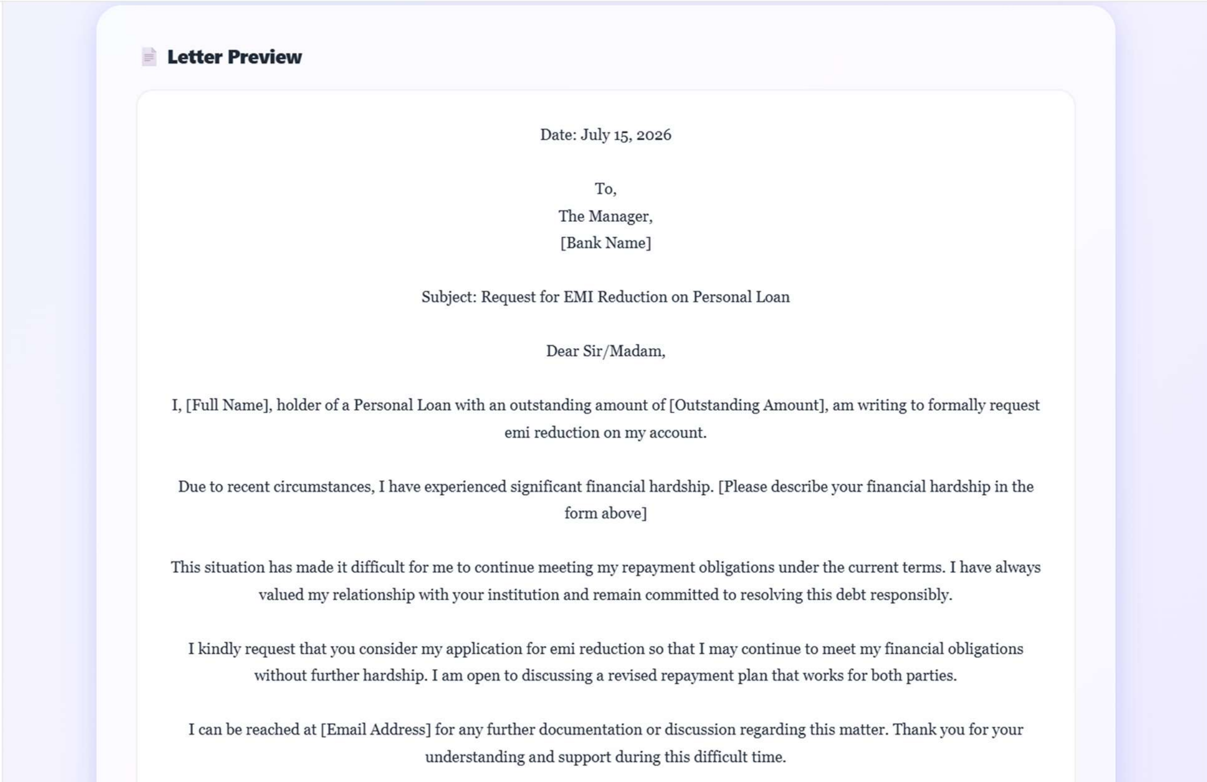
Description:

The Debt Analysis page enables users to enter their personal and financial information, including monthly income, monthly expenses, and total debt. Upon clicking the Analyze Debt button, the system calculates the debt ratio, disposable income, and financial risk level. Based on these calculations, personalized financial recommendations are displayed to help users understand their financial position and improve debt **management.**



Description:

The AI Debt Negotiation Letter Generator allows users to create a professional debt negotiation letter by entering details such as full name, email, bank name, loan type, outstanding amount, requested relief, and reason for financial hardship. After clicking the Generate AI Letter button, the application generates a structured debt negotiation letter. Users can also download or copy the generated letter for future use.



Letter Preview

Date: July 15, 2026

To,
The Manager,
[Bank Name]

Subject: Request for EMI Reduction on Personal Loan

Dear Sir/Madam,

I, [Full Name], holder of a Personal Loan with an outstanding amount of [Outstanding Amount], am writing to formally request emi reduction on my account.

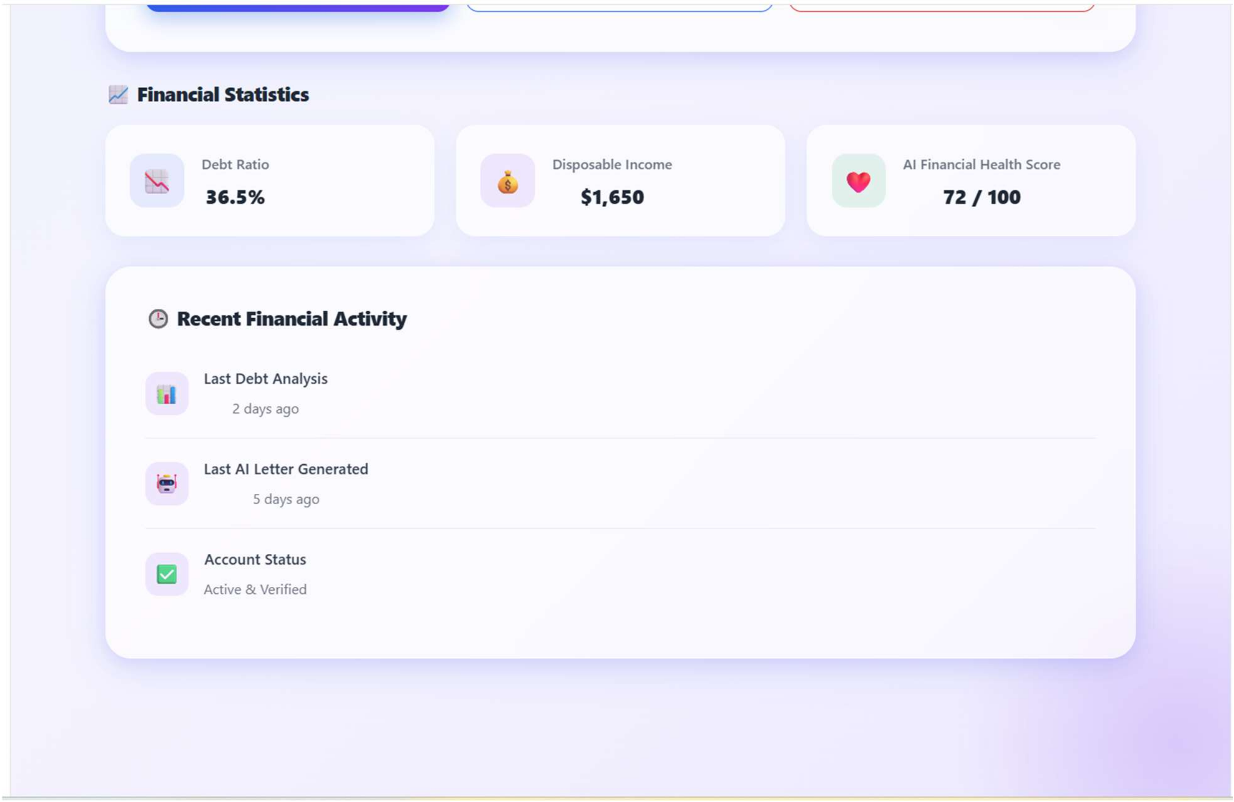
Due to recent circumstances, I have experienced significant financial hardship. [Please describe your financial hardship in the form above]

This situation has made it difficult for me to continue meeting my repayment obligations under the current terms. I have always valued my relationship with your institution and remain committed to resolving this debt responsibly.

I kindly request that you consider my application for emi reduction so that I may continue to meet my financial obligations without further hardship. I am open to discussing a revised repayment plan that works for both parties.

I can be reached at [Email Address] for any further documentation or discussion regarding this matter. Thank you for your understanding and support during this difficult time.

Description:
The Letter Preview section displays the generated debt negotiation letter in a well-formatted document layout. It includes the user's financial details and a formal request addressed to the bank, requesting debt relief or loan restructuring. This feature allows users to review the letter before copying or downloading it, ensuring accuracy and professionalism.



Conclusion:

The AI Powered Debt Relief & Financial Recovery Platform was successfully developed using React, FastAPI, and SQLite to provide users with an efficient financial assistance system. The platform supports secure user registration, debt analysis, personalized financial recommendations, and automated debt negotiation letter generation through an intuitive user interface. The application was thoroughly tested in the local development environment, and all core functionalities performed successfully. The project demonstrates the effective integration of modern web technologies to deliver a reliable, user-friendly, and scalable financial recovery solution.